CS5187: Vision & Image

香港城市大學(東莞)
City University of Hong Kong
(Dongguan)

Zhu Zhiyu

Tutorial 03: Feature Detection

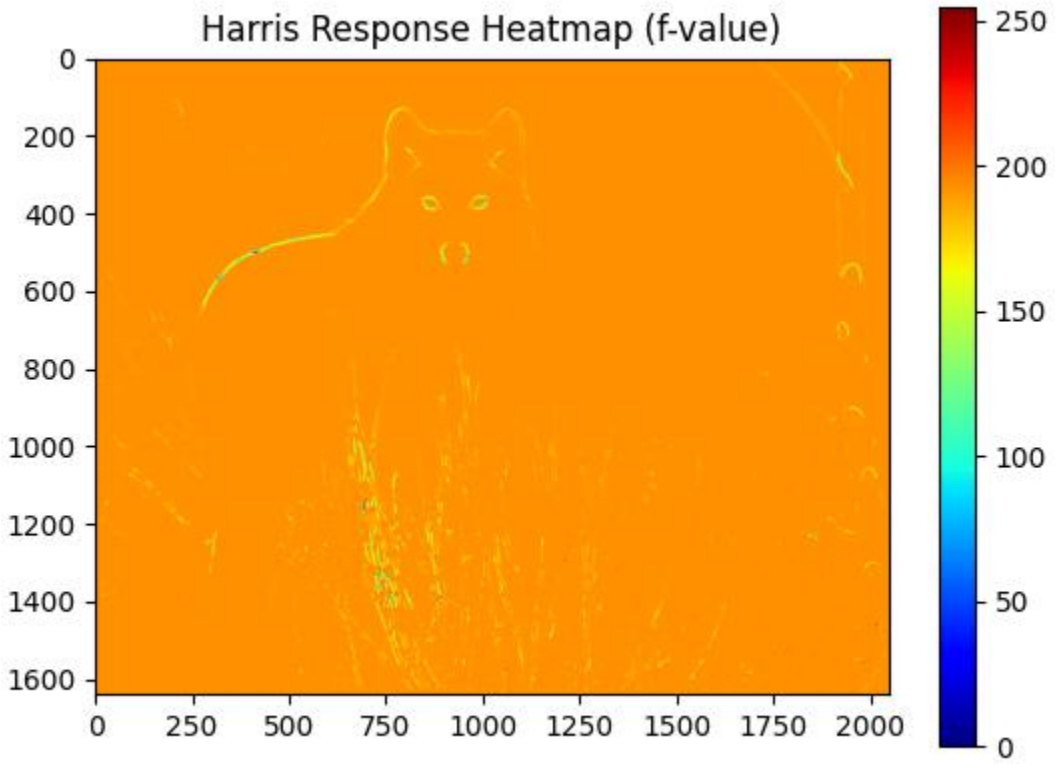
（1）Implementing the Harris Corner Response

Objective: Manually compute the Harris response function to identify interest points.

Experiment:

- Compute image derivatives I_x and I_y using Sobel filters.
- Construct the second moment matrix H by computing the weighted sums of I_x^2, I_y^2, and $I_x I_y$ using a Gaussian window.
- Calculate the cornerness score $R = \det(H) - k \cdot (\text{trace}(H))^2$ for each pixel.
- **Visualization**: Display the R values as a heatmap where red represents high cornerness and blue represents low cornerness.

（1）Implementing the Harris Corner Response




```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

def harris_response(img, k=0.04, window_size=21):
    # 1. Image derivatives using Sobel
    Ix = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=31)
    Iy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=31)

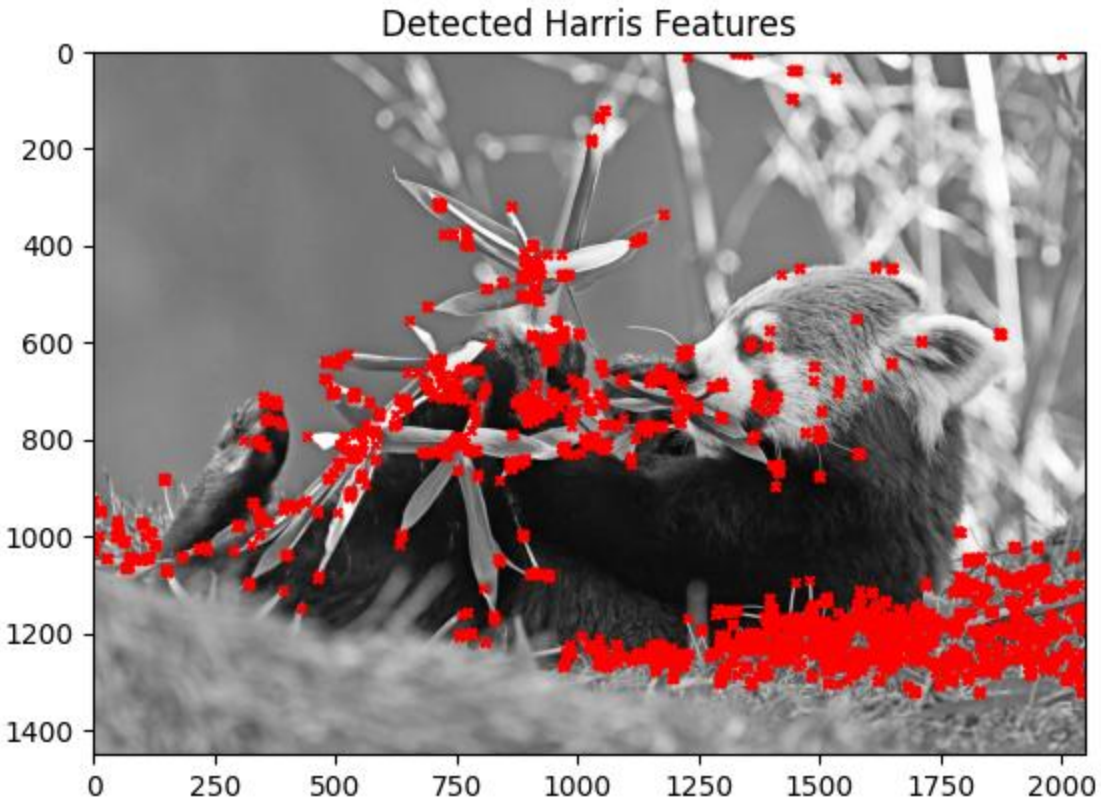
    # 2. Square of derivatives
    Ix2 = Ix**2
    Iy2 = Iy**2
    Ixy = Ix * Iy

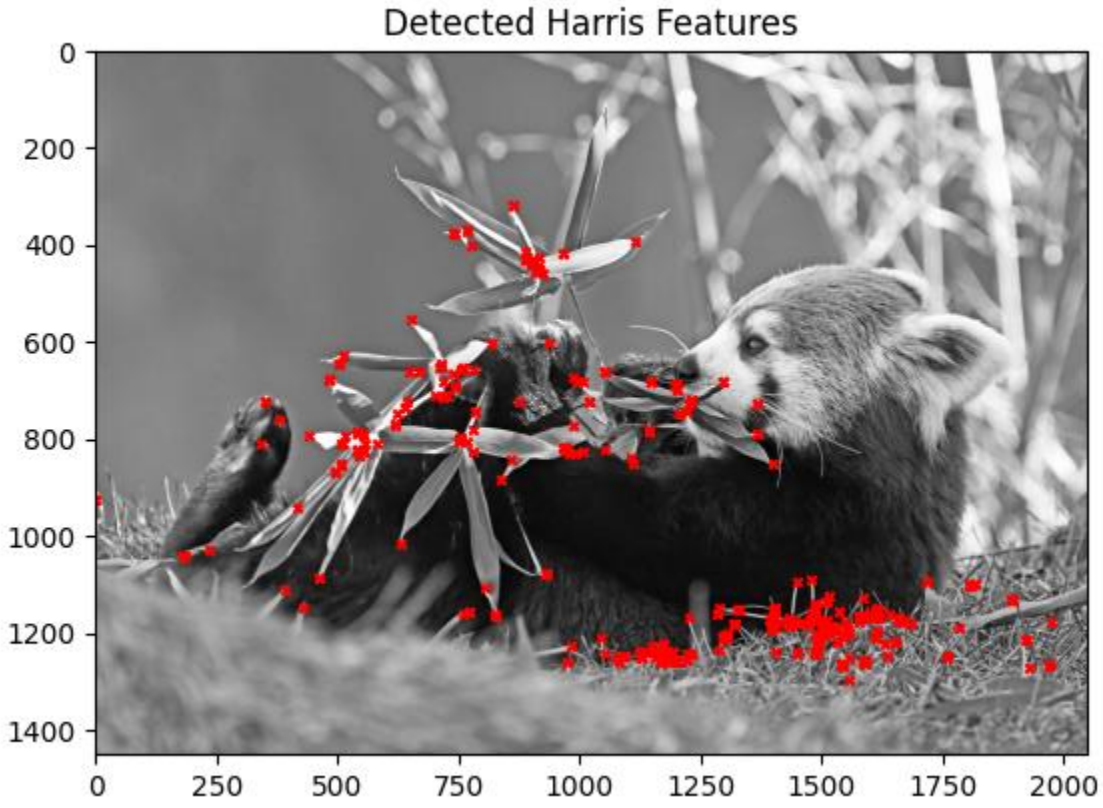
    # 3. Gaussian filter to weight the derivatives
    Sxx = cv2.GaussianBlur(Ix2, (window_size, window_size), 1)
    Syy = cv2.GaussianBlur(Iy2, (window_size, window_size), 1)
    Sxy = cv2.GaussianBlur(Ixy, (window_size, window_size), 1)

    # 4. Cornerness function: R = det(H) - k * (trace(H))^2 [cite: 23, 24]
    det_H = (Sxx * Syy) - (Sxy**2)
    trace_H = Sxx + Syy
    R = det_H - k * (trace_H**2)
    # threshold = 0.05 * R.max()
    R_visual = cv2.normalize(R, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
    return R_visual

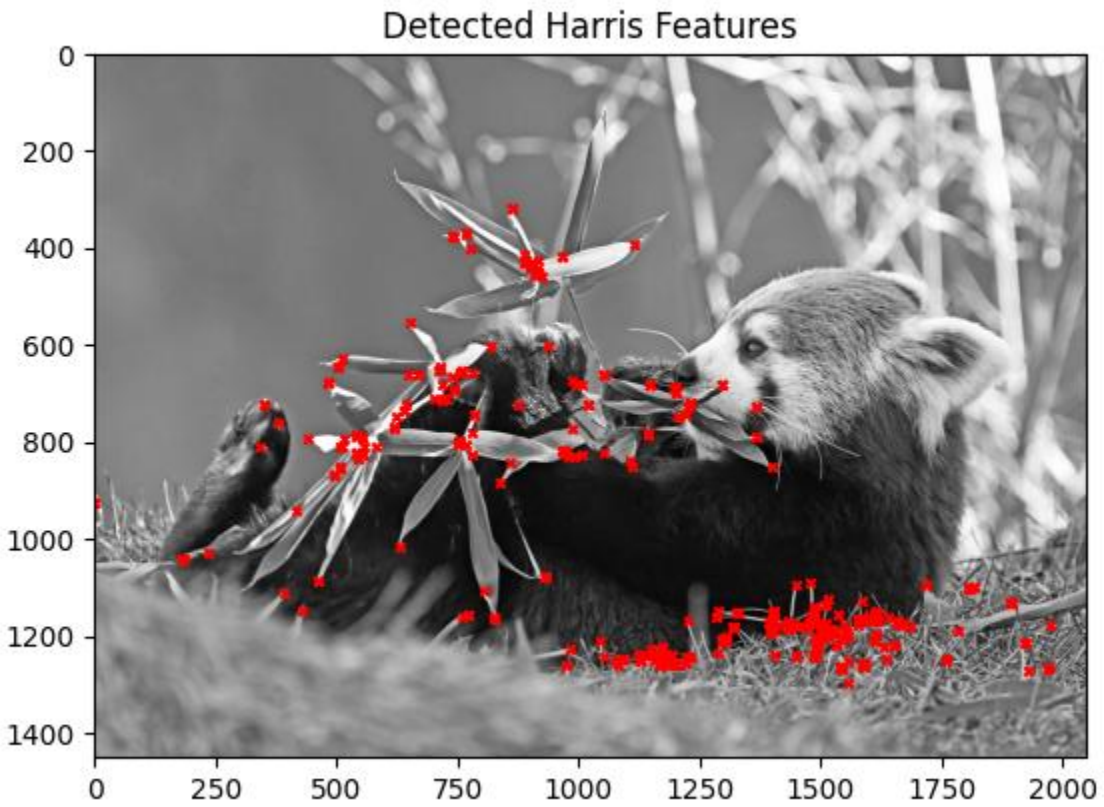
# Load and visualize the heatmap
img = cv2.imread('example_image_02.jpg', 0)
R_map = harris_response(img)
plt.imshow(R_map, cmap='jet')
plt.title("Harris Response Heatmap (f-value)")
plt.colorbar()
plt.show()
```

（2）Non-Maximum Suppression (NMS) and Thresholding





（2）Non-Maximum Suppression (NMS) and Thresholding




```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

def harris_response(img, k=0.04, window_size=21):
    # 1. Image derivatives using Sobel
    Ix = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=31)
    Iy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=31)

    # 2. Square of derivatives
    Ix2 = Ix**2
    Iy2 = Iy**2
    Ixy = Ix * Iy

    # 3. Gaussian filter to weight the derivatives
    Sxx = cv2.GaussianBlur(Ix2, (window_size, window_size), 1)
    Syy = cv2.GaussianBlur(Iy2, (window_size, window_size), 1)
    Sxy = cv2.GaussianBlur(Ixy, (window_size, window_size), 1)

    # 4. Cornerness function: R = det(H) - k * (trace(H))^2 [cite: 23, 24]
    det_H = (Sxx * Syy) - (Sxy**2)
    trace_H = Sxx + Syy
    R = det_H - k * (trace_H**2)
    # threshold = 0.05 * R.max()
    R_visual = cv2.normalize(R, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
    return R_visual

# Load and visualize the heatmap
img = cv2.imread('example_image.jpg', 0)
R_map = harris_response(img)
# plt.imshow(R_map, cmap='jet')
# plt.title("Harris Response Heatmap (f-value)")
# plt.colorbar()
# plt.show()

def detect_corners(R, threshold_ratio=0.72):
    # Thresholding: Keep only values above a certain percentage of max R
    threshold = threshold_ratio * R.max()
    R_thresholded = np.copy(R)
    R_thresholded[R < threshold] = 0

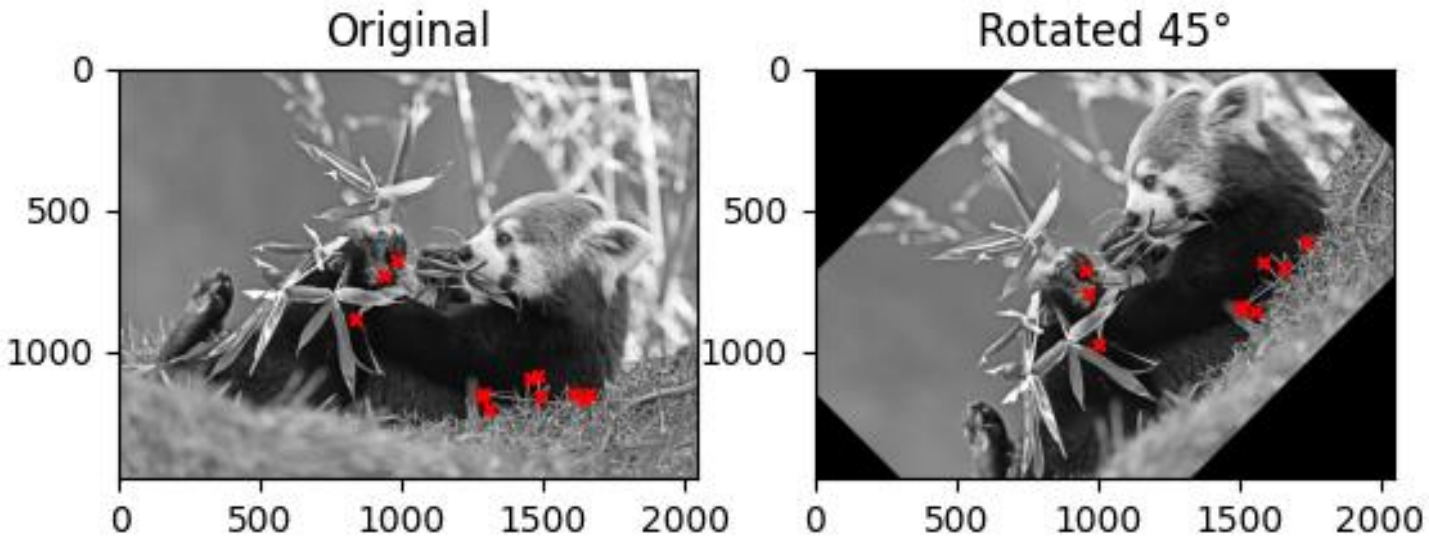
    # Non-maximum suppression: find local maxima
    # Dilate replaces each pixel with the maximum in its neighborhood
    local_max = cv2.dilate(R_thresholded, None)
    corners = (R_thresholded == local_max) & (R_thresholded > 0)

    return corners

# Detect and plot
corners = detect_corners(R_map)
y, x = np.where(corners)
plt.imshow(img, cmap='gray')
plt.scatter(x, y, s=10, c='red', marker='x')
plt.title("Detected Harris Features")
plt.show()
```

You can change the ratio

（3）Invariance Testing: Rotation




```python
import numpy as np
import matplotlib.pyplot as plt

def harris_response(img, k=0.04, window_size=5):
    # Use a smaller ksize for local gradients
    Ix = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
    Iy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

    Ix2, Iy2, Ixy = Ix**2, Iy**2, Ix*Iy

    # Use a smaller window (e.g., 5x5) for better localization
    Sxx = cv2.GaussianBlur(Ix2, (window_size, window_size), 0)
    Syy = cv2.GaussianBlur(Iy2, (window_size, window_size), 0)
    Sxy = cv2.GaussianBlur(Ixy, (window_size, window_size), 0)

    det_H = (Sxx * Syy) - (Sxy**2)
    trace_H = Sxx + Syy
    R = det_H - k * (trace_H**2)

    return R # Return raw float values, don't normalize yet!

# Load and visualize the heatmap
img = cv2.imread('example_image.jpg', 0)
R_map = harris_response(img)
# plt.imshow(R_map, cmap='jet')
# plt.title("Harris Response Heatmap (f-value)")
# plt.colorbar()
# plt.show()

def detect_corners(R, threshold_ratio=0.7):
    # Thresholding: Keep only values above a certain percentage of max R
    threshold = threshold_ratio * R.max()
    R_thresholded = np.copy(R)
    R_thresholded[R < threshold] = 0

    # Non-maximum suppression: find local maxima
    # Dilate replaces each pixel with the maximum in its neighborhood
    local_max = cv2.dilate(R_thresholded, None)
    corners = (R_thresholded == local_max) & (R_thresholded > 0)

    return corners

def rotate_image(image, angle):
    (h, w) = image.shape[:2]
    center = (w // 2, h // 2)
    M = cv2.getRotationMatrix2D(center, angle, 1.0)
    return cv2.warpAffine(image, M, (w, h))

# Compare original vs rotated
img_rotated = rotate_image(img, 45)
R_rotated = harris_response(img_rotated)
corners = detect_corners(R_map)
corners_rotated = detect_corners(R_rotated)

# Visualization: Students should see that corners "follow" the rotation [cite: 253]
plt.subplot(121), plt.imshow(img, cmap='gray'), plt.title("Original")
y, x = np.where(corners)
plt.scatter(x, y, s=10, c='red', marker='x')
plt.subplot(122), plt.imshow(img_rotated, cmap='gray'), plt.title("Rotated 45°")
y, x = np.where(corners_rotated)
plt.scatter(x, y, s=10, c='red', marker='x')
plt.show()
```

Questions?

香港城市大學（東莞）
City University of Hong Kong
(Dongguan)